

# Metodología de Sistemas II

## Trabajo Práctico 1

### Gobernanza y Definición de Terminado (DoD)

## 1. Redaccion de la Definition of Done

Para que cada incremento de software de nuestro equipo sea realmente entregable, se establece la siguiente Definition of Done formal. Ningun Pull Request podra fusionarse a la rama principal si no cumple la totalidad de los seis criterios que se detallan a continuacion.

En primer lugar, integridad de compilacion: el codigo debe compilar sin errores ni advertencias criticas en el entorno de Integracion Continua, con la rama actualizada respecto de la principal y sin conflictos de fusion. En segundo lugar, validacion de pruebas automatizadas: todas las pruebas unitarias y de integracion del modulo modificado deben ejecutarse y pasar en estado verde, y no se aceptan pruebas omitidas o ignoradas sin una justificacion tecnica aprobada. En tercer lugar, umbrales minimos de cobertura de codigo: las lineas nuevas o modificadas deben mantener o superar el 80 por ciento de cobertura, y el reporte generado por la pipeline no debe evidenciar caidas en el porcentaje global del proyecto. En cuarto lugar, revision por pares obligatoria: todo cambio requiere la aprobacion explicita de al menos un desarrollador distinto al autor, quien confirma que la logica es correcta, que no existen vulnerabilidades de seguridad evidentes y que se respetan los patrones de diseño del proyecto. En quinto lugar, gobernanza de estilo: el codigo debe superar las herramientas de analisis estatico y formateo automatico configuradas en el repositorio, de modo que la sintaxis, la nomenclatura y la estructura visual sean homogeneas en toda la base de codigo. Por ultimo, sustento de la documentacion tecnica: todo cambio que altere la interfaz publica, la arquitectura o el comportamiento del sistema debe acompañarse de la actualizacion de la documentacion correspondiente, como README, diagramas o comentarios de cabecera, ya que no se considera terminado un desarrollo que el resto del equipo no pueda entender y utilizar.

## 2. Configuracion de trazabilidad: plantilla de incidencia

Se propone la siguiente plantilla profesional en formato Markdown, diseñada para estandarizar el reporte de problemas y eliminar la ambigüedad desde el primer contacto. Al copiarla al documento, conserva los simbolos tal cual, porque ellos son justamente el formato Markdown que pide la consigna:

```
---
name: Reporte de Incidencia Tecnica
about: Utilice esta plantilla para reportar un error o comportamiento inesperado en el sistema.
title: "[BUG] "
labels: bug
assignees: ""
---
```

### Titulo descriptivo del problema

(Reemplazar este texto con un resumen claro y conciso del fallo. Ejemplo: El sistema arroja error 500 al intentar guardar un paciente sin obra social).

#### ### Descripcion tecnica del problema

Detalle de manera objetiva que esta fallando en el sistema. Incluya el modulo, servicio o componente afectado si lo conoce.

#### ### Pasos exactos para reproducir el fallo

Por favor, detalle la secuencia exacta de acciones para que el equipo de desarrollo pueda replicar el error en un entorno local:

1. Ir a la seccion de...
2. Ingresar los siguientes datos...
3. Hacer clic en el boton...
4. Observar el resultado...

#### ### Comportamiento esperado

Describa que deberia haber sucedido segun los requisitos del sistema o la logica de negocio.

#### ### Comportamiento observado

Describa que sucedio realmente. Incluya mensajes de error exactos, codigos de estado HTTP o comportamientos erraticos de la interfaz.

#### ### Evidencias adjuntas

(Pegue aqui capturas de pantalla, registros de logs del servidor, trazas de error del navegador o videos cortos que demuestren el fallo).

#### ### Entorno tecnico

- Version de la aplicacion o rama de Git:
- Sistema Operativo y Navegador (si aplica):
- Base de datos o entorno especifico (Desarrollo, Pruebas, Produccion):

### **3. Matriz de categorizacion: sistema de etiquetas**

Para gestionar el ciclo de vida del desarrollo y auditar el cumplimiento de la DoD antes de integrar codigo, se propone un sistema de cuatro etiquetas. La etiqueta bug identifica un defecto o comportamiento inesperado en codigo existente; su justificacion tecnica es que permite priorizar correcciones por sobre nuevas funcionalidades y, durante la auditoria de la DoD, exige verificar que se haya creado un test de regresion que evite la reaparicion del fallo. La etiqueta technical-debt señala tareas de refactorizacion, actualizacion de dependencias obsoletas o limpieza que no aportan funcionalidad visible pero sostienen la salud del proyecto; se audita contra los criterios de gobernanza de estilo, cobertura y revision por pares, garantizando que la refactorizacion no introduzca defectos nuevos. La etiqueta documentation agrupa la creacion o mejora de manuales, README, comentarios o

especificaciones de API, y permite verificar de forma inmediata el criterio de sustento de documentacion de la DoD antes del merge. Por ultimo, la etiqueta feature corresponde a nuevas funcionalidades o historias de usuario solicitadas por el negocio; un Pull Request con esta etiqueta solo supera la auditoria si cumple de manera estricta los umbrales de cobertura y la validacion completa de las pruebas automatizadas, demostrando que la logica nueva es robusta y estable.